Master in Computer Vision Barcelona

C5 Project: Multimodal Recognition

Session 3

Image Captioning (1)

Carlos Boned Riera
cboned@cvc.uab.cat

Ernest Valveny
ernest@cvc.uab.cat

Lei Kang
lkang@cvc.uab.cat

Image Captioning Overview

Image Captioning is the process of generating descriptive textual sentences that convey the content of an image. This technique integrates both **computer vision (CV)** and **natural language processing (NLP)** to automatically analyze visual data and produce coherent, contextually relevant captions that reflect the objects, actions, and scenes depicted in images.

A happy dog is standing in the ocean.

Image Captioning Datasets

VizWiz-Captions Dataset

https://vizwiz.org/tasks-and-datasets/image-captioning/

A computer screen with a Windows message about Microsoft license terms.

A can of green beans is sitting on a counter in a kitchen.

A photo taken from a residential street in front of some homes with a stormy sky above.

A blue sky with fluffy clouds, taken from a car while driving on the highway.

A hand holds up a can of Coors Light in front of an outdoor scene with a dog on a porch.

A digital thermometer resting on a wooden table, showing 38.5 degrees Celsius.

A Winnie The Pooh character high chair with a can of Yoohoo sitting on it in front of a white wall.

A cup holder in a car holding loose change from Canada.

- Training images: 23,431 Training captions: 117,155
- Validation images: 7,750 Validation captions: 38,750
- Test images: 8,000 Test captions: 40,000

Image Captioning Datasets

VizWiz-Captions Dataset

https://vizwiz.org/tasks-and-datasets/image-captioning/

A computer screen with a Windows message about Microsoft license terms.

A can of green beans is sitting on a counter in a kitchen.

A photo taken from a residential street in front of some homes with a stormy sky above.

A blue sky with fluffy clouds, taken from a car while driving on the highway.

A hand holds up a can of Coors Light in front of an outdoor scene with a dog on a porch.

A digital thermometer resting on a wooden table, showing 38.5 degrees Celsius.

A Winnie The Pooh character high chair with a can of Yoohoo sitting on it in front of a white wall.

A cup holder in a car holding loose change from Canada.

- **Our training set (90% for training and 10% for validation)**
  - Training images: 23,431 Training captions: 117,155
- **Our test set**
  - Validation images: 7,750 Validation captions: 38,750
- ~~Test images: 8,000 Test captions: 40,000~~

Image Captioning Baseline Models

Recall the image classification process:

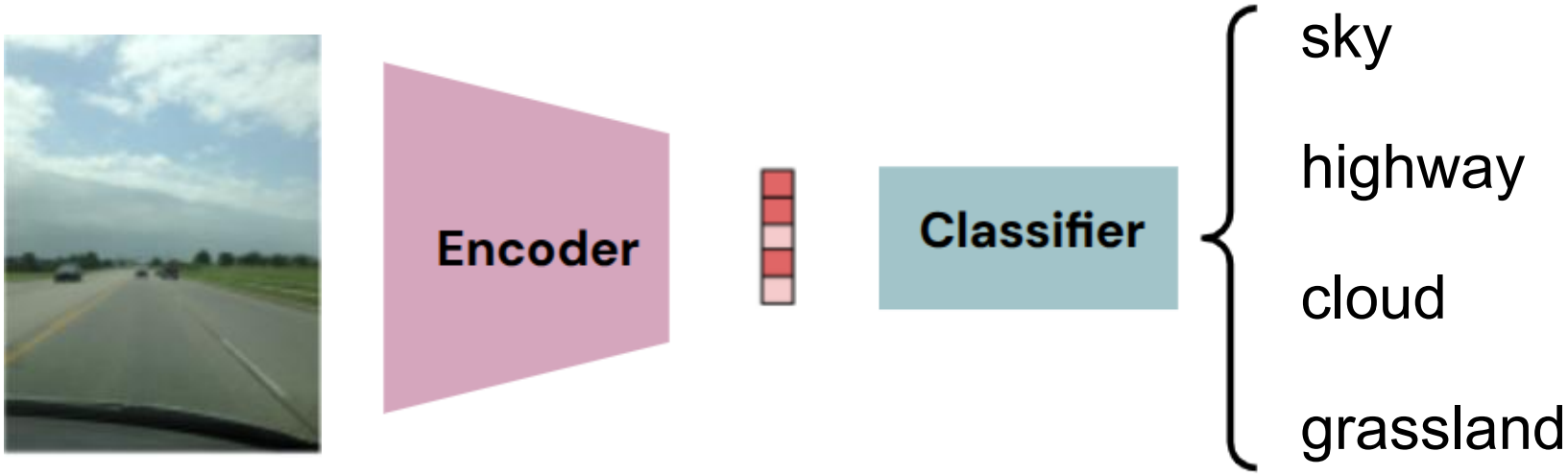


Image Captioning

More classification heads?

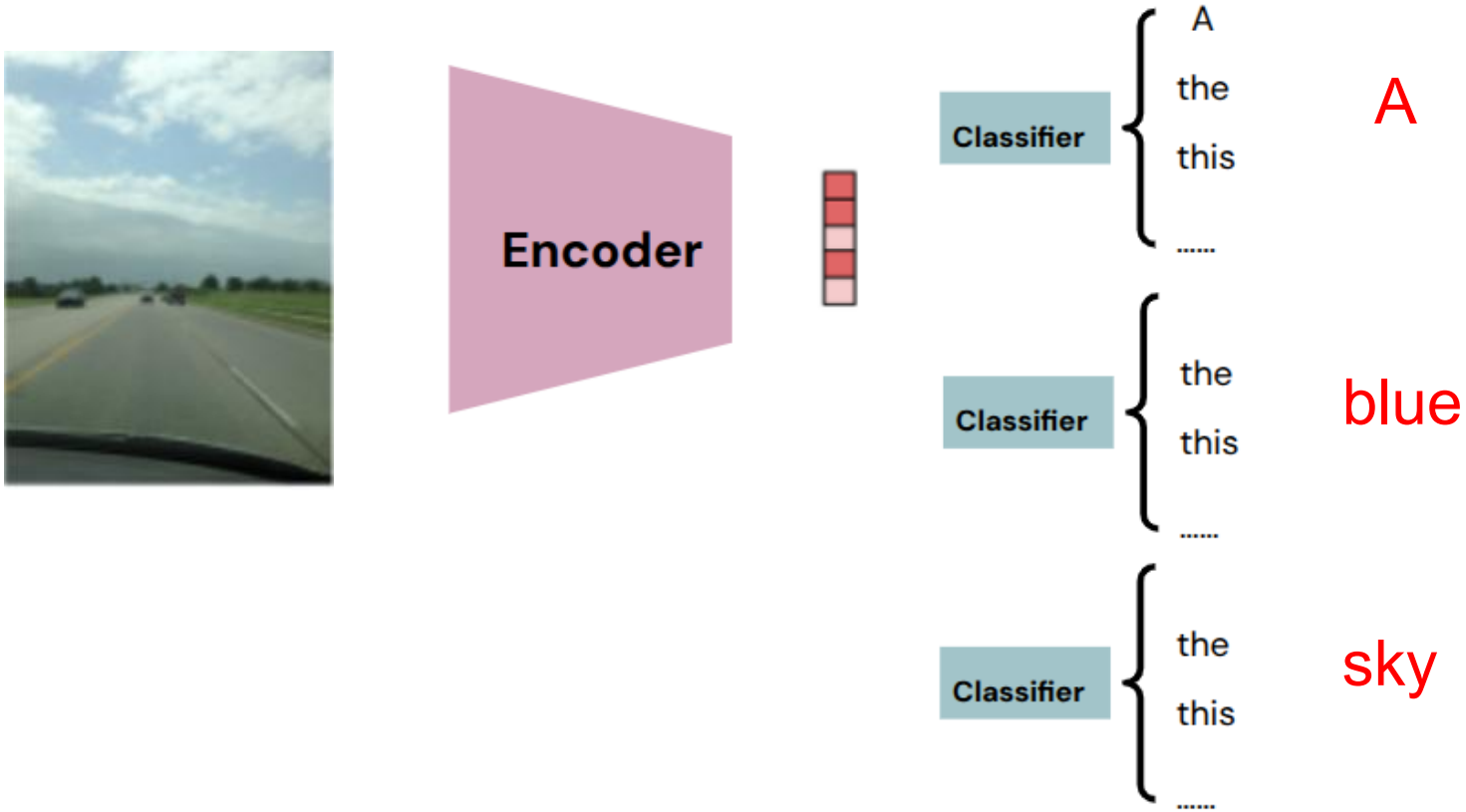


Image Captioning

Question: How can we add connections between classification heads to ensure contextual consistency?

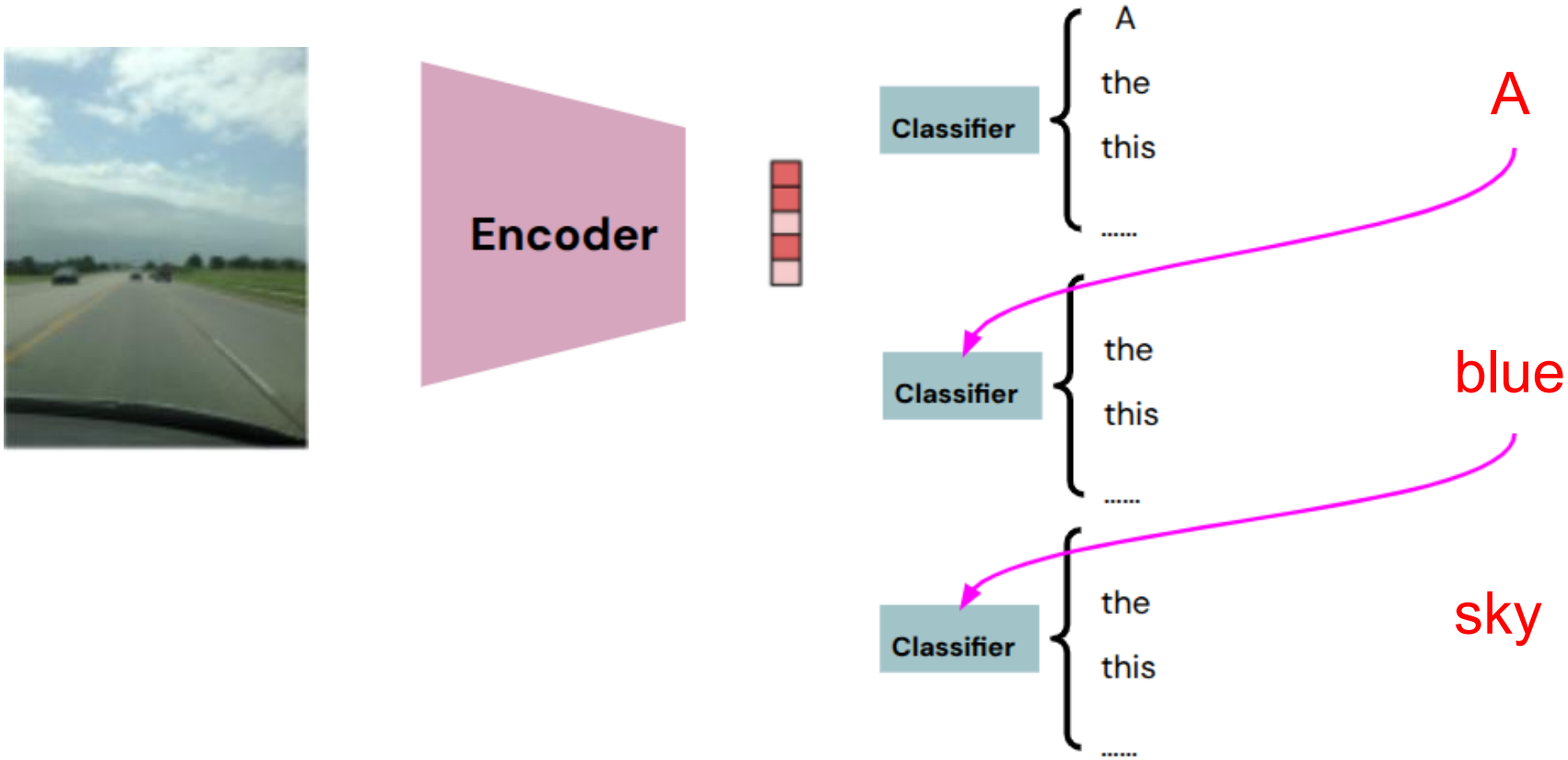


Image Captioning

Solution: We can achieve this by leveraging RNNs, commonly used in NLP tasks.

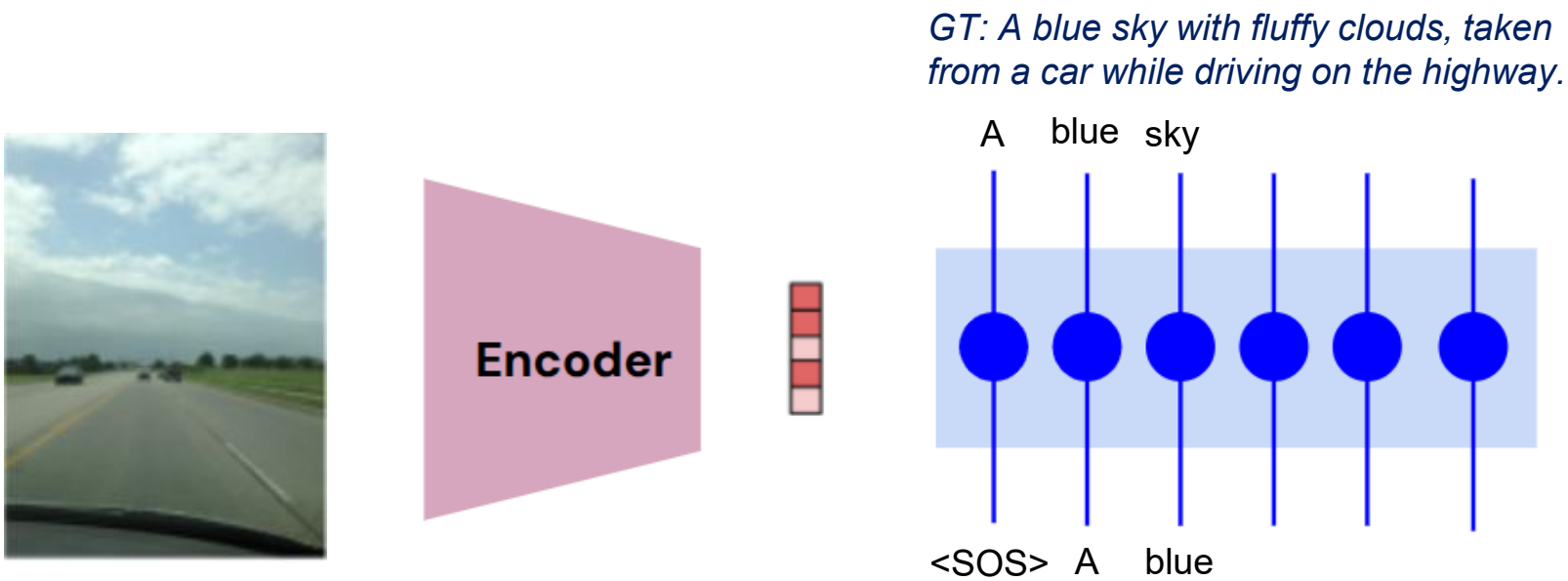
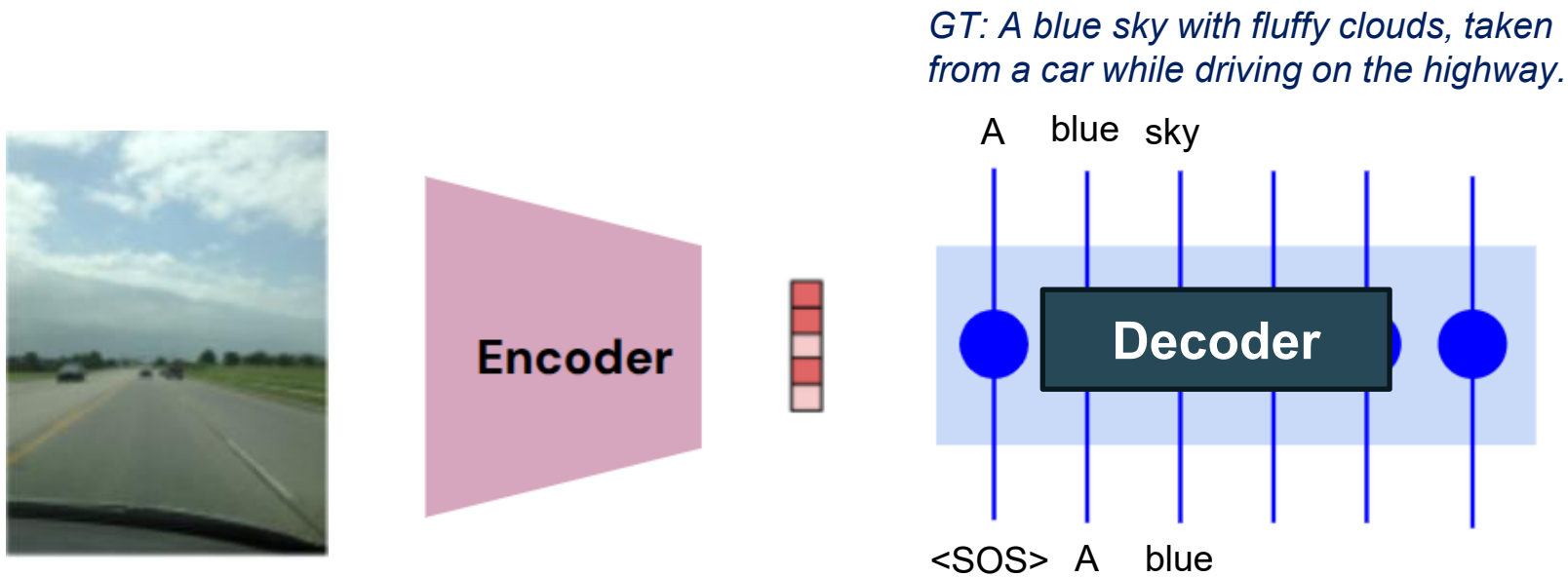


Image Captioning

Here is the classic architecture: Encoder-Decoder.



Question 1: which model CAN be used as an Encoder for image captioning?
Question 2: which model CAN be used as a Decoder for image captioning?
Question 3: Which decoding level is better: character, sub-word, or word?

Image Captioning: Baseline Model (Encoder-Decoder)

Encoder: ResNet-18
Decoder: GRU

"Baseline Model and Metrics.ipynb" (in Campus Virtual)

Tasks:

- **Encoder Modifications:** Replace ResNet-18 with alternatives like ResNet-34, ResNet-50, or VGG16.
- **Decoder Modifications:** Experiment with different architectures, such as LSTM, and vary the number of layers.
- **Text Representation Levels:** Implement wordpiece-level, or word-level representations.
- **Optimizer Variations:** Test different optimizers, including SGD, Adam, and AdamW.
- **Training Strategies:** Adjust learning rate, implement learning rate decay, apply early stopping, and experiment with teacher forcing.

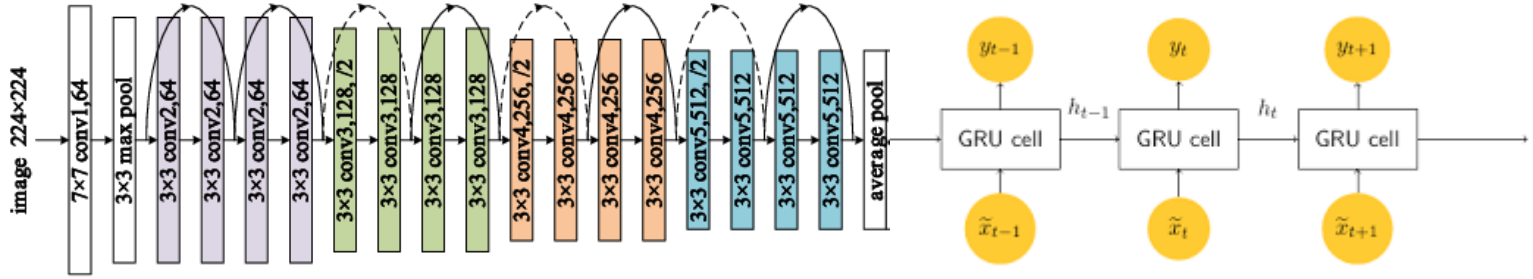


Image Captioning: Text Representation Levels

1. **Character-level (baseline)**
   - Treat captions as sequences of individual characters.
     - Pros: Simpler vocabulary, handles OOV words effectively.
     - Cons: Longer sequences, harder to learn long-range dependencies.
2. **Subword-level**
   - Captions represented as subword units (e.g. Byte-Pair Encoding, SentencePieceTokenizer, pre-trained tokenizers of LLMs, ...).
     - Pros: Balances vocabulary size and flexibility; effectively handles unknown words.
     - Cons: Requires pre-processing to determine optimal subwords.
3. **Word-level**
   - Captions represented as sequences of complete words.
     - Pros: Simple, intuitive, shorter sequences.
     - Cons: Large vocabulary; difficulties handling OOV words.

Text Representation Levels Examples

1. **Character-level (baseline)**
   - "A dog running on grass"
     - [A, , d, o, g, , r, u, n, n, i, n, g, , o, n, , g, r, a, s, s]
     - Vocabulary size very small (26 – 100 characters)
2. **Subword-level**
   - "A dog running on grass"
     - [A] [dog] [run] [ning] [on] [grass]
     - Algorithms: Byte Pair Encoding, WordPiece, or SentencePiece
     - Vocabulary size (1k – 30k tokens)
3. **Word-level**
   - "A dog running on grass"
     - [A] [dog] [running] [on] [grass]
     - Vocabulary size (10k – 50k words)

Image Captioning: Metrics

- BLEU (Bilingual Evaluation Understudy)
- ROUGE (Recall-Oriented Understudy for Gisting Evaluation)
- METEOR(Metric for Evaluation of Translation with Explicit Ordering)

Image Captioning: Metrics -- BLEU

BLEU score measures the similarity between the machine-translated text and the reference translations using n-grams, which are contiguous sequences of n words.

$$\text{BLEU} = \min\left(1, \frac{\textit{output-length}}{\textit{reference-length}}\right) \left(\prod_{i=1}^{4} \textit{precision}_i\right)^{\frac{1}{4}}$$

Brevity Penalty is a penalty term that adjusts the score for translations that are shorter than the reference translations.

Precision of n-grams, which is calculated as the number of n-grams that appear in both the machine-generated translation and the reference translations divided by the total number of n-grams in the machine-generated translation.

Image Captioning: Metrics – BLEU Example

Reference: “I go to class” Candidate: “Go to class”

Unigrams:

Candidate	Reference	Match?
Go	go	Yes
to	to	Yes
class	class	Yes

- **Precision:** 3 matches / 3 candidate unigrams = **1.0**

Image Captioning: Metrics – BLEU Example

Reference: “I go to class” Candidate: “Go to class”

Bigrams:

Candidate	Reference	Match?
Go to	go to	Yes
to class	to class	Yes

- **Precision:** 2 matches / 2 candidate bigrams = **1.0**

Image Captioning: Metrics – BLEU Example

Reference: “I go to class” Candidate: “Go to class”

Trigrams:

Candidate	Reference	Match?
Go to class	go to class	Yes

- **Precision:** 1 match / 1 candidate trigram = **1.0**

Image Captioning: Metrics – BLEU Example

Reference: “I go to class” Candidate: “Go to class”

4-grams:
Not applicable (Candidate length is less than 4)

Image Captioning: Metrics – BLEU Example

Reference: “I go to class” Candidate: “Go to class”

Brevity Penalty (BP):

Candidate length = 3 words
Reference length = 4 words
BP = exp(1 - 4/3) ≈ 0.7165

BLEU-1: 0.7165, BLEU-2: 0.7165, BLEU-3: 0.7165, BLEU-4: 0

Image Captioning: Metrics – BLEU Code

```
Import evaluate
bleu = evaluate.load('bleu')
bleu1 = bleu.compute(predictions=pred1, references=ref, max_order=1)
bleu2 = bleu.compute(predictions=pred1, references=ref, max_order=2)
```

Image Captioning: Metrics -- ROUGE

ROUGE score calculates the recall of n-grams in the machine-generated summary by comparing them to the reference summaries.

Rouge-N (F1) measures the overlap of n-grams

- Precision = (No. of overlapping n-grams/ Total no. of n-grams in the generated summary)
- Recall = (No. of overlapping n-grams/ Total no. of n-grams in the reference summary)
- F1-score = 2*(precision * recall)/ (precision + recall)

Rouge-L measures the longest subsequence between the generated summary and reference summary. A subsequence is a sequence of words that appear in the same order, but not necessarily consecutively.

ROUGE-Lsum splits the text into sentences based on newlines and computes the LCS for each pair of sentences, while ROUGE-L ignores newlines and computes the LCS for the entire text. This is appropriate when the candidate and reference summaries contain a single long sequence.

Image Captioning: Metrics – ROUGE Example

Reference: “I go to class” Candidate: “Go to class”

Matching Unigrams:

Unigram	In Reference	In Candidate	Match
go	Yes	Yes	Yes
to	Yes	Yes	Yes
class	Yes	Yes	Yes
I	Yes	No	No

- **Number of Matching Unigrams:** 3

Calculations:

- **Precision:** $\frac{3}{3} = 1.0$
- **Recall:** $\frac{3}{4} = 0.75$
- **F1 Score:** $2 \times \frac{1.0 \times 0.75}{1.0 + 0.75} = 0.8571$

Image Captioning: Metrics – ROUGE Example

Reference: “I go to class” Candidate: “Go to class”

Matching Bigrams:

Bigram	In Reference	In Candidate	Match
go to	Yes	Yes	Yes
to class	Yes	Yes	Yes
I go	Yes	No	No

- **Number of Matching Bigrams:** 2

Calculations:

- **Precision:** $\frac{2}{2} = 1.0$
- **Recall:** $\frac{2}{3} \approx 0.6667$
- **F1 Score:** $2 \times \frac{1.0 \times 0.6667}{1.0 + 0.6667} \approx 0.8$

Image Captioning: Metrics – METEOR

METEOR score is based on the harmonic mean of unigram precision and recall, with recall weighted higher than precision.

- m: the number of mapped unigrams found between the two strings
- t: the total number of unigrams in the translation
- r: the total number of unigrams in the reference
- ch: number of chunks (contiguous matched sequences)

Unigram precision P=m/t, unigram recall R=m/r
default weighting with α=0.9
Penalty = $\gamma \cdot (ch/m)^{\beta}$

$$\textbf{METEOR} = (1 - \textbf{Penalty}) \cdot F_{mean}$$

$$F_{mean} = \frac{P \cdot R}{\alpha \cdot P + (1-\alpha) \cdot R}$$

β: a parameter responsible for the penalty's shape, default 3
Gamma γ: the relative weight for the fragmentation penalty, default 0.5

Image Captioning: Metrics – METEOR Example

Reference: “I go to class” Candidate: “Go to class”

Total Matches (M): 3, Chunks (C): 1 (All words align in order: “go to class”)

Candidate Position	Candidate Word	Reference Position	Reference Word	Match Type
1	Go	2	go	Exact
2	to	3	to	Exact
3	class	4	class	Exact

Precision = 3/3 = 1.0, Recall = 3/4 = 0.75, F_mean = (1*0.75)/(0.9*1+0.1*0.75) = 0.7692
Penalty = 0.5 * (1/3)^3 = 0.0185185
METEOR = 0.7692 * (1 - 0.0185185) = 0.7549

Image Captioning: Metrics – Code

https://github.com/huggingface/evaluate

Evaluate

pip install nltk
pip install rouge_score
pip install evaluate

https://huggingface.co/spaces/evaluate-metric/bleu
https://huggingface.co/spaces/evaluate-metric/meteor
https://huggingface.co/spaces/evaluate-metric/rouge

VizWiz

VizWiz-Captions Dataset

https://vizwiz.org/tasks-and-datasets/image-captioning/ (to download the sets)
https://github.com/Yinan-Zhao/vizwiz-caption (to process the json files)

Image Captioning: Metrics Usage in this Project

- **BLEU-1**
- **BLEU-2**
- **ROUGE-L**
- **METEOR**

```
ref = [['A child is running in the campus']]
pred1 = ['A child campus']

bleu1 = bleu.compute(predictions=pred1, references=ref, max_order=1)
bleu2 = bleu.compute(predictions=pred1, references=ref, max_order=2)
res_r = rouge.compute(predictions=pred1, references=ref)
res_m = meteor.compute(predictions=pred1, references=ref)

f"BLEU-1:{bleu1['bleu']*100:.1f}%, BLEU2:{bleu2['bleu']*100:.1f}%, ROUGE-L:{res_r['rougeL']*100:.1f}%, METEOR:{res_m['meteor']*100:.1f}%"
```

```
'BLEU-1:26.4%, BLEU2:18.6%, ROUGE-L:60.0%, METEOR:38.7%'
```

C5 Project Stages and Schedule

Session 1 February 16	Task 1.a: Object detection
Session 2 March 4	T1.b: Object Segmentation
Session 3 March 16	Task 2: Image Captioning (1)
Session 4 March 23	Task 2: Image Captioning (2)
Session 5 April 13	Task 3: Diffusion models (1)
Session 6 April 20	Task 3: Diffusion models (2)
Session 7 April 27	Final presentation

EASTER HOLIDAYS (March 30 – April 6)

Deliverable: Report Image Captioning (1)

Deliverable: Report Image Captioning (2)

Image Captioning Tasks [Week 1]

1. **Modify the baseline** model and implement your own models:
   a. Change the encoder choosing **ONE** alternative **encoder** module: ResNet-34/50, VGG-16/19, …
   b. Change the decoder choosing **ONE** alternative **decoder** module: LSTM, xLSTM, …
   c. Change the **text representation** level among **subword-level**, and **word-level**
   d. [Optional] Try to implement an attention mechanism

   - Include in your slides the description of the models

2. **Train and evaluate** your models. Compare among them and with the baseline. Show quantitative and qualitative results and analyze the impact of each individual modification to the baseline.

   - Include in your slides quantitative and qualitative results

Due date

March 23rd, Monday, before 10:00 AM

Deliverable (for next follow-up session)

- **Github** repository with readme.md (code explanation & instructions)
- **Presentation** with all items listed in the tasks. Include **one** summary slide at the end of your presentation with main results and conclusions
  - One member of the group will have to present this slide in **1 minute** during the follow-up session.